

Code Variation as a Defense

University of Texas at Dallas
Erick Bauman - exb131030@utdallas.edu
Huibo Wang - hwx142830@utdallas.edu
Travis Wright - tnw130030@utdallas.edu

March 31, 2018

Abstract

This research seeks to determine whether obfuscating the structure of binaries and systems through compiler and kernel modification has any benefit to security

Index terms— Code Variation, Obfuscation, Compiler Alteration, Kernel Alteration

1 Project Summary

1.1 Project Description

The goal of this project is to determine if security can be increased by obfuscating the layout of a binary file or the system itself. By changing the compiler settings, the compiler code, or the kernel code, we will observe changes in binary files and attempt to determine if these changes are sufficient to increase security.

1.2 Team Responsibilities

- Set up a Linux environment, using Ubuntu, with Tiny C Compiler
- Make Modification to Tiny C Compiler
- Build a web server application such as Nginx using the modified compiler and test its functionality and stability
- Analyze the binary for degree of difference as a metric

1.3 Total Budget

An estimated 10 hours a week will be devoted to the project

1.4 Deliverables

- Proposal and literature review
- Progress report and presentation
- Final report and presentation
- A modified version of Tiny C Compiler along with documentation

2 Executive Summary

Title: Code Variation as a Defense

Affiliation: University of Texas at Dallas

Authors: Erick Bauman - exb131030@utdallas.edu

Huibo Wang - hwx142830@utdallas.edu

Travis Wright - tnw130030@utdallas.edu

Index terms— Code Variation, Obfuscation, Compiler Alteration, Kernel Alteration

Summary: The goal of our project is to test the effectiveness of code variation as a defense. By changing the code of a compiler or kernel, we will attempt to change the system in order to thwart or complicate current attacks as well as maintain correct behavior and stability. By studying the amount of variation that we create, its stability and correctness, and its resiliency to modern attacks, we can measure if this research was effective. We will be starting small, working with the Tiny C Compiler and a single web server application, like Nginx, to apply and study the changes.

3 Motivation

The layout of a system is in most cases well defined. A little research can teach an individual what each register is generally used for, the calling convention of a system, the system call numbers, and the even the rough layout of memory. This knowledge is leveraged by attackers to pivot within a system that has been breached. We set out to study the usefulness of obfuscating these known facts. By changing these common conventions, we seek to secure the system as a whole. If changes to the compiler or kernel can alter a system enough while preserving stability and functionality, then many modern attacks that rely on some assumption regarding the base system will fail.

4 Literature Review

4.1 *How to Make ASLR Win the Clone Wars: Runtime Re-Randomization* [3]

RuntimeASLR is a proposed solution to clone-forking attacks, which leverage the lack of re-randomization of memory after a fork within a process. The application performs taint analysis to find all module pointers within the program. Once the fork is executed and the pointers are found, the modules are shuffled in memory and the recorded pointers are remapped to point to the new address of the originally pointed to module. Evaluation of the application applied to Nginx show that it can detect all pointers are remap them in a way to preserve functionality and prevent clone-fork attacks with the cost of some extra overhead.

4.2 *CodeArmor: Virtualizing the code space to counter disclosure attacks* [2]

CodeArmor is a proposed tool that seeks to defeat code disclosure attacks such as Return-Oriented Programming. It virtualizes the code space, decoupling the pointer values from any concrete locations. In addition, honey gadgets are placed throughout the program to trick attackers and bait them into using invalid gadgets that lead them out of usable memory. CodeArmor also does not need access to hypervisors or source code, simply running as an application that attaches to binaries. It is proven to raise resiliency to code disclosure attacks with a 14.5% overhead for common web server applications.

4.3 *Smashing the Stack for Fun and Profit* [4]

This explains buffer overflow attacks and how exploits against binary code are performed. A programmer mistake in a program can lead to the potential of a buffer overflow, which can be used to write content outside of its intended location. This paper discusses stack overflows, in which

a buffer overflow on the stack allows an attacker to overwrite return addresses with values the attacker chooses, and therefore allows the attacker to redirect control to any destination. At the time of this attack, the stack was both writable and executable, meaning that the attacker could write arbitrary bytes onto the stack and overwrite the return address to return directly into those bytes. This *shellcode* can perform arbitrary computation since the attacker can fill the stack with any bytes they choose.

4.4 *The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)* [5]

With the introduction of $W \oplus X$, direct code injection became impossible. However, control flow could still be redirected to existing code in the binary. This introduces an attack against binary applications that is now known as Return-Oriented Programming, which is an extension of earlier attacks that alter a program's control flow to perform attacker-chosen actions. By stringing together known gadgets, or a small series of instructions generally followed by a return instruction, the attacker can overwrite memory and jump to wherever they please. This power allows the attacker to completely hi-jack the control flow of the program. The attacker can then perform almost any action they wish.

4.5 *Hacking Blind* [1]

Hacking Blind introduces a new kind of attack called Blind Return-Oriented Programming. It abuses programs that fork new processes, such as the majority of web server applications, and causes them to leak data one bit at a time based on what input causes the program to crash. Since fork does not re-randomize memory, every fork will have the same memory layout and gadgets can be collected with no knowledge of the underlying program that is being attacked. The collected gadgets can then be leveraged to attack the system much as a traditional Return-Oriented Programming attack would. This attack defeats modern Address Space Layout Randomization and Data Execution Prevention.

5 Specific Aims

The goal of this project is determine the feasibility of code variation as a defense. Our research will focus specifically on using and modifying Tiny C Compiler while building Nginx. After alterations to the compiler are made, we will build Nginx and test that the functionality is unaltered and the program is still stable. We will then study the degree of variation that was achieved both in the binary and in system memory. Our scope will be limited to memory disclosure attacks. We seek to make dumping memory more difficult and make attacks like Return-Oriented Programming difficult or even impossible. If our research produces positive results, then there is promise in such a technique being applied to additional programs or to a system itself.

6 Plans

The group will meet weekly to ensure progress and redirect the research if needed. Effective time management and communication will also ensure the project progresses smoothly and on schedule.

7 Deliverables

- Proposal and literature review
- Progress report and presentation
- Final report and presentation
- A modified version of Tiny C Compiler along with documentation

8 Progress

So far, we have decided that we will be making changes to Tiny C Compiler in an attempt to randomize the address space layout. We have cloned the source code for TCC and begun studying it.

9 Going Forward

We plan to continue trying to understand the source code of TCC so that we can make changes to it. Once we have a solid understanding of the source, we will be attempting to randomize the address space layout but shifting the order of data on the stack as well as potentially changing the default behavior for each register.

10 Issues

Thus far in the project, the primary obstacle has been the poorly documented TCC source code. It is surprisingly difficult to understand what is actually happening with no comments and no documentation to explain what the code does and why it is performing the actions it is.

The primary obstacle of this research will be the preservation of correct behavior and stability. When making changes to the compiler or the kernel, it would be easy to alter the behavior of programs. By making the changes incrementally, we hope to minimize the amount of errors introduced and be able to revert changes quickly when needed.

Another large obstacle will be creating enough variation to prevent attacks. With the existence of attacks such as the Blind ROP, which makes no assumptions on the underlying system, it will be difficult to alter the system in such a way that prevents the attack. With attacks like Blind Return-Oriented Programming, the scope of our project prevents us from attempting to mitigate it. Our approach here is simply trial and error. Applying changes and one after the other and testing along the way.

The final issue we may face is uncovering unintended bugs in programs. If a program makes any assumptions about the underlying system, then introducing code variation could cause these assumptions to break the program. In a case like this, maintaining correct behavior or stability might be impossible.

References

- [1] A. Bittau, A. Belay, A. Mashtizadeh, D. Mazières, and D. Boneh. Hacking blind. In *2014 IEEE Symposium on Security and Privacy*, pages 227–242, May 2014.
- [2] X. Chen, H. Bos, and C. Giuffrida. Codearmor: Virtualizing the code space to counter disclosure attacks. In *Security and Privacy (EuroS&P), 2017 IEEE European Symposium on*, pages 514–529. IEEE, 2017.
- [3] K. Lu, W. Lee, S. Nürnberger, and M. Backes. How to make aslr win the clone wars: Runtime re-randomization. In *NDSS*, 2016.
- [4] A. One. Smashing the stack for fun and profit. *phrack magazine*, 49 (14), nov. 1996.
- [5] H. Shacham. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86). In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 552–561. ACM, 2007.

11 Biographical Sketches of the Investigators

11.1 Erick Bauman

Erick Bauman holds a Bachelor’s degree in Computer Science and is currently pursuing a PhD in Computer Science. He is a research assistant in the Lab for Systems and Software Security at the University of Texas at Dallas, and is researching new techniques for binary rewriting and its

applications. He has completed coursework in systems security, advanced programming languages, computer networks, and operating systems.

11.2 Huibo Wang

Huibo Wang holds a Master's degree in Information System and is currently pursuing a PhD in Computer Science. She is a research assistant in the Lab for Systems and Software Security at the University of Texas at Dallas, and is researching new techniques for System security, especially SGX related problems. She has completed coursework in systems security, operating system, computer networks, wireless network, machine learning, artificial Intelligence.

11.3 Travis Wright

Travis Wright holds a Bachelor's degree in Computer Science and is currently pursuing a Master's in Computer Science specializing in Information Assurance. He is a recipient of the National Science Foundation's CyberCorps: Scholarship for Service and completed an internship with the Department of Defense as a Security Analyst. He has completed coursework in computer networks, wireless networks, network security, information security, and operating systems. He is an associate of (ISC)² toward CISSP.

12 Schedule

Feb 9	Submit Proposal
Feb 16	Set Up Environment & Record Initial Benchmark
Feb 23	Test Changes to Compiler Flags
Mar 2	Progress Report & Presentation
Mar 9	Test Changes to Compiler Flags
Mar 16	Compiler Modification
Mar 23	Compiler Modification
Mar 30	Compiler Modification
Apr 6	Compiler Modification
Apr 13	Compiler Modification
Apr 20	Compiler Modification
Apr 27	Final Report & Presentation

13 Budget

Our project has no monetary costs. The group is aiming for 10 hours of work a week in addition to our weekly meetings.